

OBLIGATORIO BD1 2026: Informe de Decisiones de Diseño

Integrantes del equipo: Virginia Martínez y María Inés Barral

1. Introducción

Este informe describe las principales decisiones de diseño tomadas por el equipo durante el desarrollo del sistema de gestión de actividades deportivas universitarias, abarcando el modelo entidad-relación, el esquema relacional en MySQL, la estructura del backend en python utilizando FastAPI y la estructura del frontend en React.

2. Entidades del modelo

El sistema cuenta con las siguientes entidades: Facultad, Carrera, Estudiante, Usuario, Disciplina, Espacio, Actividad, Practica, Inscripcion y Asistencia. Para ver las relaciones entre entidades y los atributos de cada una de ellas, ver el modelo entidad-relación adjunto en el repositorio del proyecto.

3. Decisiones de diseño

3.1 Separación entre Actividad y Practica

Se distingue entre Actividad (la definición general: disciplina, espacio, horario y días) y Practica (la ocurrencia concreta de esa actividad en una fecha específica). Esta separación permite que una misma Actividad genere múltiples Practicas a lo largo del tiempo, reflejando el modelo de un club deportivo donde las clases se repiten semana a semana.

3.2 Cardinalidad Practica — Inscripcion: 1 a 1

La decisión más relevante del grupo fue modelar la inscripción a nivel de Practica individual, no a nivel de Actividad. Es decir, un Estudiante se inscribe a cada clase (Practica) de forma separada.

Esto implica que la relación entre Practica e Inscripcion, para un mismo Estudiante, es de uno a uno: una Practica tiene a lo sumo una Inscripcion por Estudiante, y cada Inscripcion corresponde a exactamente una Practica.

Las razones que llevaron a esta decisión fueron:

- Control granular de asistencia: al vincular la inscripción a una Practica concreta, el registro de asistencia se simplifica, ya que Asistencia referencia directamente a Inscripcion, que a su vez ya contiene la fecha exacta de la clase.
- Cupos por clase: permite controlar cupo máximo y mínimo a nivel de Practica, no solo de Actividad.
- Cancelaciones puntuales: si una clase específica se cancela, solo afecta las inscripciones de esa Practica sin impactar el resto.
- Integridad a nivel de base de datos: la tabla Asistencia tiene un UNIQUE INDEX sobre id_inscripcion, lo que garantiza que no puede existir más de un registro de asistencia por inscripción, reforzando el 1:1 directamente en el esquema.

3.3 Usuario vinculado a Estudiante

Usuario es una entidad separada de Estudiante. Tiene un FK opcional hacia Estudiante (id_estudiante), que puede ser NULL. La regla es: un usuario con rol 'admin' no tiene estudiante asociado (id_estudiante debe ser NULL), y un usuario con rol 'estudiante' debe tener siempre un Estudiante vinculado (id_estudiante NOT NULL). Esta restricción se controla con un CHECK constraint en la base de datos y se valida también en la capa de servicio del backend.

El campo de login del Usuario es email, que es único e independiente del email del Estudiante.

3.4 Días de Actividad como ENUM

El campo día en la tabla Actividad se implementó como ENUM con valores fijos: Lunes, Martes, Miercoles, Jueves, Viernes, Lunes y Miercoles, Martes y Jueves, Miercoles y Viernes. Esto evita inconsistencias en la carga y refleja los patrones reales de horarios universitarios.

3.5 Borrado lógico

Todas las entidades principales tienen un campo activo (TINYINT, default 1). Se optó por borrado lógico en lugar de eliminación física para preservar la integridad referencial y mantener historial de datos.

4. Estructura del backend

El backend está implementado en Python con FastAPI, organizado en capas: routes (endpoints HTTP), services (lógica de negocio y validaciones), repositories (acceso a la base de datos) y schemas (modelos Pydantic para validación). La conexión a MySQL usa mysql-connector-python. Las contraseñas de los usuarios se almacenan hasheadas con bcrypt. La fundamentación de utilizar los patrones de diseño service y repository es separar responsabilidades entre diferentes clases, logrando un código más escalable.

5. Control de versiones

El proyecto se gestiona en GitHub con ramas por funcionalidad. El archivo database/schema.sql representa el estado limpio y final del esquema.